

One-to-One Relationships in Entity Framework Core

In this article, I will discuss how to configure **One-to-One Relationships between Two Entities in Entity Framework Core (EF Core)** with Examples. Please read our previous article on [Relationships Between Entities in EF Core](#).

What is a One-to-One Relationship in Entity Framework Core?

A One-to-One (1:1) relationship in Entity Framework Core represents a scenario where one entity is associated with exactly one other entity. These relationships are used when a single record in one table is linked to only one record in another table.

For example, consider an application that manages user and their corresponding passport. Each User has exactly one Passport, and each Passport belongs to exactly one User. This is a classic One-to-One relationship.

Guidelines to Implement One-to-One Relationships in Entity Framework Core

The following are the key Guidelines to Implement One-to-One Relationships in Entity Framework Core:

- **Principal and Dependent Entities:** In a One-to-One relationship, one entity is the principal, and the other is the dependent. So, first, decide which entity is the Principal and which entity is the dependent.
- **Foreign Key Constraints:** The dependent entity typically contains the foreign key.
- **Optional vs. Required Relationship:** Define whether the relationship is optional or required. If it's optional, the dependent entity can exist without the principal entity.
- **Database Schema:** In the database, One-to-One relationships are often represented by a **shared primary key** or a foreign key constraint where the **foreign key is unique**.

Real-Time Example to Understand One-to-One Relationships in EF Core

Consider an example of **User and Passport** entities to demonstrate the implementation of One-to-One relationships in Entity Framework Core. Here, I will show you how to implement One-to-One Relationships in EF Core with the following three approaches:

- **Implementing One-to-One Relationships Without Data Annotations or Fluent API**
- **Implementing One-to-One Relationships with Data Annotations**
- **Implementing One-to-One Relationships with Fluent API**

Implementing One-to-One Relationships Without Data Annotations or Fluent API

EF Core can sometimes manage relationships automatically without any explicit configuration. However, if we don't use data annotations or Fluent API, EF Core will attempt to create default relationships based on naming conventions. Let's understand how to implement One-to-One Relationships with default EF Core conventions.

Creating Entities:

In our example, we want to establish One-to-One Relationships between User and Passport entities. So, we need to create these two entities.

User Entity

So, create a class file named **User.cs** within the Entities folder and then copy and paste the following code. This is going to be our Principal entity.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class User
    {
        public int Id { get; set; }
        public string Username { get; set; }
        public Passport Passport { get; set; } // Navigation property
    }
}
```

Passport Entity

Next, create a class file named **Passport.cs** within the Entities folder and copy and paste the following code. This will be our Dependent entity, and here, we will create the unique foreign key column.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Passport
    {
        public int Id { get; set; }
        public string PassportNumber { get; set; }
        public int UserId { get; set; } //FK, Required Relationship
        public User User { get; set; } // Navigation property
    }
}
```

With Default Convention:

- EF Core can detect the One-to-One relationship based on the presence of navigation properties and foreign key properties.
- The Passport entity has a foreign key UserId and a navigation property User.
- The User entity has a navigation property, Passport.

Key Entities and Their Roles

- **Principal Entity (User):** The main entity in the relationship. In this case, the User is the principal entity because the relationship is established around each user having one passport.
- **Dependent Entity (Passport):** This is the entity that depends on the principal entity. The Passport is the dependent entity because it has a foreign key (UserId) that references the Primary Key of the User entity.

DbContext Class:

Next, modify the EFCoreDbContext class as follows. Here, we are adding both User and Passport as DbSet properties so that EF Core can generate the required database tables with a One-to-One relationship between them.

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
```

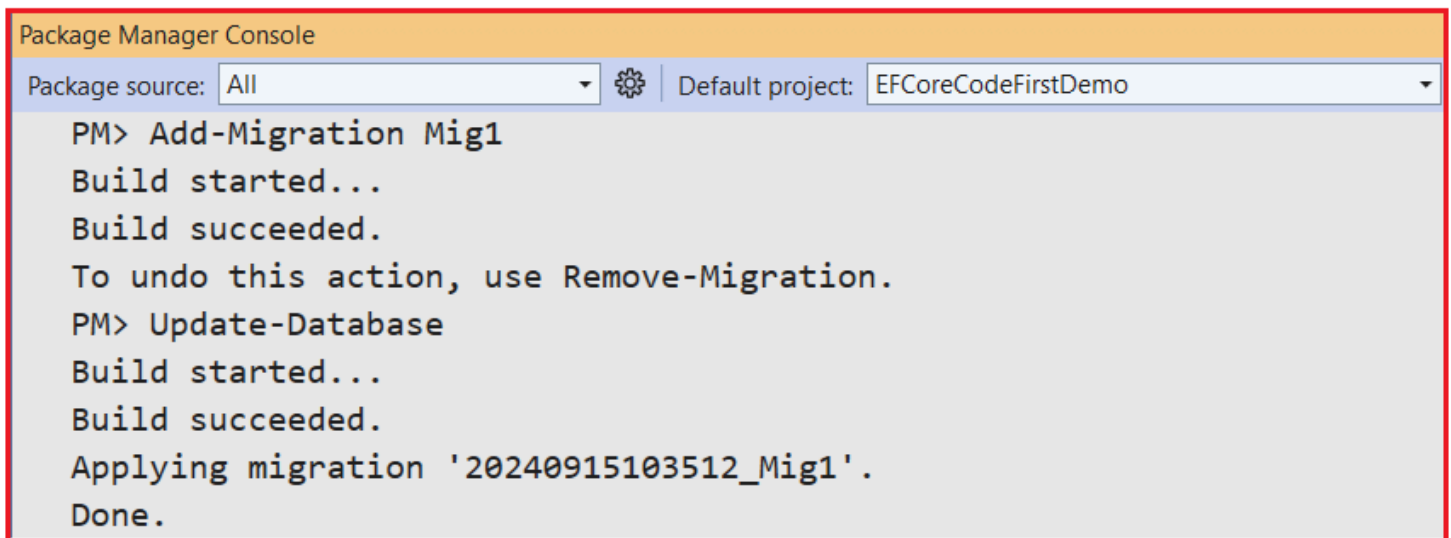
```

        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            //Configuring the Connection String
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=PassportDB;Trusted_Connection=True;TrustServerCertificate=True;");
        }
        public DbSet<User> Users { get; set; }
        public DbSet<Passport> Passports { get; set; }
    }
}

```

Generating the Migration:

With the above changes, open the Package Manager Console and Execute the Add-Migration and Update-Database commands as follows.



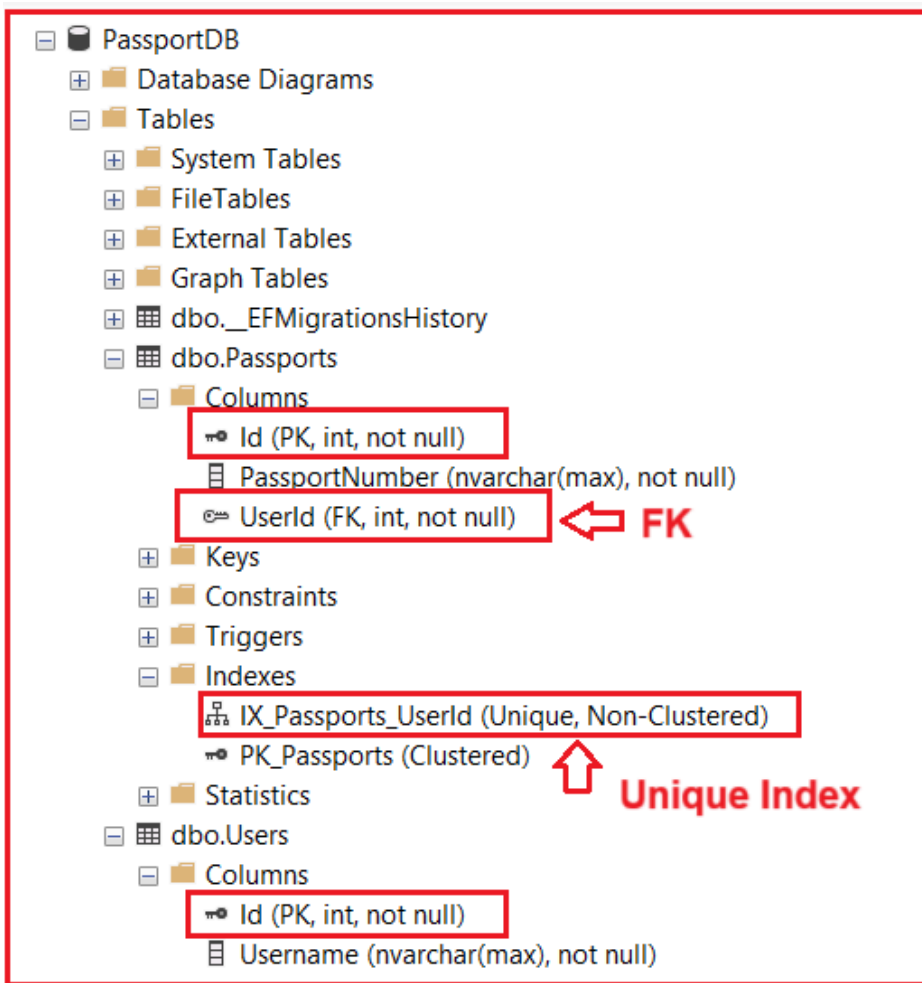
The screenshot shows the Package Manager Console window with the following content:

```

Package Manager Console
Package source: All [v] [gear] Default project: EFCoreCodeFirstDemo [v]
PM> Add-Migration Mig1
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> Update-Database
Build started...
Build succeeded.
Applying migration '20240915103512_Mig1'.
Done.

```

Once you execute the above commands, the database should be created with the Required Users and Passports tables with One-to-One relationships. In the Passports table, the foreign key is created on the UserId column, which points to the Users table Primary Key column.



Further, if you verify the Passport table, you will also see that it applies the Unique Index on the UserId column. The unique index allows the column value to be unique, and the value must be an existing UserId, hence implementing a One-to-One Relationship.

Limitations of this approach:

- Without explicit configuration, EF Core might misinterpret the relationship or may not enforce the One-to-One constraints properly.
- It's recommended to use Data Annotations or Fluent API for clarity and to ensure the relationship is correctly established.

Implementing One-to-One Relationships Using Data Annotation Attributes in EF Core

We can use Data Annotations Attributes to define One-to-One relationships explicitly between two entities in Entity Framework Core.

Entities with Data Annotations:

First, modify the User entity as follows:

```
using System.ComponentModel.DataAnnotations;
namespace EFCoreCodeFirstDemo.Entities
{
```

```

public class User
{
    [Key]
    public int Id { get; set; }
    public string Username { get; set; }
    public Passport Passport { get; set; } // Navigation property
}
}

```

Next, modify the Passport entity as follows:

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    public class Passport
    {
        [Key]
        public int Id { get; set; } //PK
        public string PassportNumber { get; set; }
        public int UserId { get; set; } //FK, Required Relationship
        [ForeignKey("UserId")]
        public User User { get; set; } // Navigation property
    }
}

```

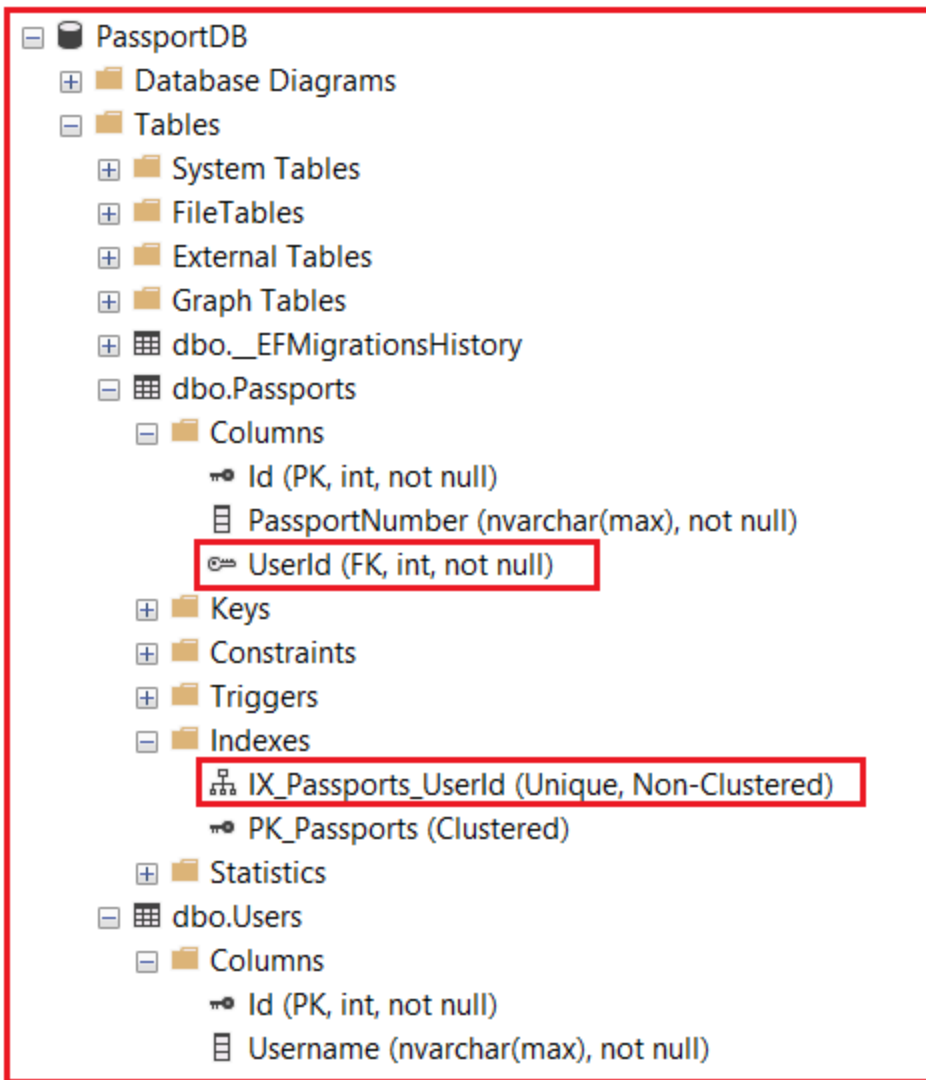
Explanation:

- In the User class, the ID is marked with the [Key] attribute, indicating that it is the Primary key in the database.
- In the Passport class, the ID is marked with the [Key] attribute, indicating that it is the Primary key in the database.
- In the Passport class, the [ForeignKey("UserId")] attribute is decorated with the User reference navigation property, indicating that the UserId column will be the foreign key referencing the User entity.

This configuration ensures that each Passport is linked to exactly one User, enforcing a one-to-one relationship.

Generating Migration:

With the above changes, open the Package Manager Console and Execute the Add-Migration and Update-Database commands. Once the commands are executed, please verify the database, and you should see the same database schema as the previous example, as shown in the below image:



This approach is simple and effective for many scenarios, especially when the relationship is straightforward.

Implementing One-to-One Relationships in EF Core Using Fluent API in EF Core

The Fluent API provides a more expressive way to configure the model within the `OnModelCreating` method of the `DbContext` class. This is also the recommended approach for complex configurations. Let us first modify the Entities by removing the `Key` and `ForeignKey` attributes from the model properties:

User:

Modify the `User` entity as follows:

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class User
    {
        public int Id { get; set; }
        public string Username { get; set; }
        public Passport Passport { get; set; } // Navigation property
    }
}
```

Passport:

Modify the Passport entity as follows:

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Passport
    {
        public int Id { get; set; } //PK
        public string PassportNumber { get; set; }
        public int UserId { get; set; } //FK, Required Relationship
        public User User { get; set; } // Navigation property
    }
}
```

Fluent API Configuration in DbContext Class:

We need to configure the one-to-one relationships between user and passport using Fluent API by overriding the OnModelCreating method in the DbContext class. You can start configuring from the Principal entity or the Dependent Entity. So, modify the EFCoreDbContext class as follows. In the example below, we start the configuration from the principal entity.

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            //Configuring the Connection String
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-6P5NK25R\SQLSERVER2022DEV;Database=PassportDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        // Override the OnModelCreating method to customize the model building
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            // Using Fluent API to define entity relationships within the OnModelCreating method
            // Start configuring the User entity
            modelBuilder.Entity<User>() //Refers to the User entity
            {
                // Specifies that the User entity has a one-to-one relationship with the Passport entity, meaning each User has one Passport.
                .HasOne(u => u.Passport)
                // Specifies that the Passport entity is also related to exactly one User entity, making the relationship bidirectional.
                .WithOne(p => p.User)
                // Sets the UserId property in the Passport entity as the foreign key for the User entity's primary key.
                .HasForeignKey<Passport>(p => p.UserId);
            }
            // Defining a DbSet for Users, representing the Users table in the database
            public DbSet<User> Users { get; set; }
        }
    }
}
```

```

        // Defining a DbSet for Passports, representing the Passports table in
        public DbSet<Passport> Passports { get; set; }
    }
}

```

Understanding Fluent API Methods:

To Implement One to One Relationship in EF Core using Fluent API, we use the following three Fluent API Methods:

- **HasOne():** Declares that the Principal entity is related to one Dependent Entity. That means it establishes that the User entity (Principal) is related to one Passport entity (Dependent).
- **WithOne():** Ensures the Dependent entity is also related to one Principal entity. It specifies the other side of the relationship, the Passport entity (dependent), which is also related to exactly one User (Principal) entity.
- **HasForeignKey():** This method defines the foreign key linking the dependent entity to the principal entity. It explicitly sets the foreign key (UserId) in the Passport entity, linking it to the Primary Key (ID) of the User entity.

Deciding Which Fluent API Methods to Use for Relationship Configuration?

When configuring relationships using the Entity Framework Core Fluent API, choosing the correct method (HasOne, HasMany, WithOne, WithMany,HasForeignKey) depends on the navigation properties of the entities and their relationship. The choice of methods depends on the following two things:

- The type of navigation properties in the entities.
- The direction of the relationship being established (principal vs. dependent entity).

How to determine which methods to use:

Starting Point: Principal Entity

- Always start with the entity that references the other entity through a navigation property.
- Use HasOne or HasMany based on whether the navigation property is:
 - Reference Navigation Property: Use HasOne.
 - Collection Navigation Property: Use HasMany.

Specify the Relationship: Dependent Entity

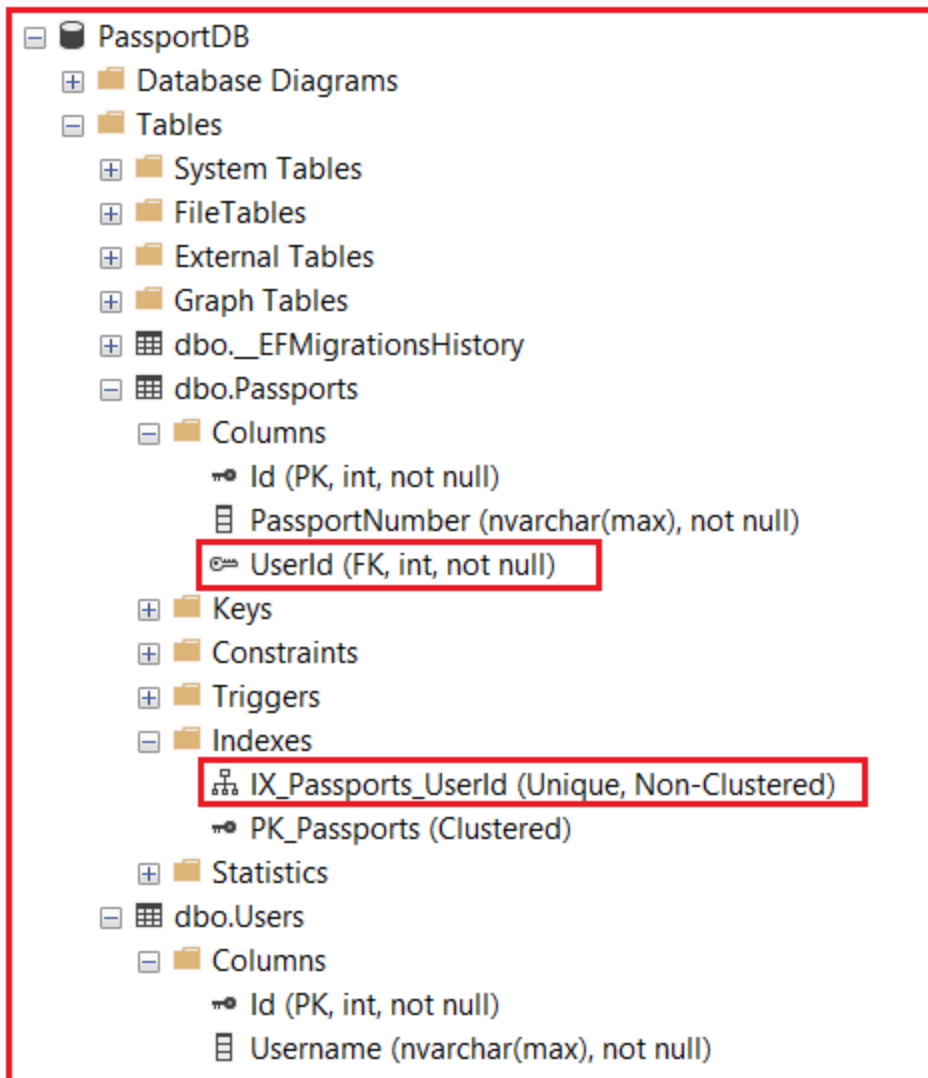
- Use WithOne or WithMany to define the relationship from the dependent entity's perspective:
 - If the dependent entity has a Reference Navigation Property, use WithOne.
 - If the dependent entity has a Collection Navigation Property, use WithMany.

Specifying the Foreign Key

Use the HasForeignKey method to explicitly specify the foreign key property in the dependent entity that links it to the principal entity. If no foreign key is specified, EF Core tries to infer the foreign key based on the default naming conventions.

Migration and Database Update:

After setting up the entities and DbContext, open the Package Manager Console and Execute the **Add-Migration** and **Update-Database** commands. Once the commands are executed, please verify the database, and you should see the same database schema as the previous example, as shown in the below image:



Shared Primary Key in a One-to-One Relationship in EF Core

One of the most efficient ways to map a one-to-one relationship in Entity Framework Core is using a shared primary key. This means that the dependent entity's primary key is also its foreign key, which references the primary key of the principal entity. When we create a One-to-One relationship with a shared primary key, we enforce a strict association between the principal and dependent entities, ensuring that:

- The dependent entity cannot exist without the principal entity.
- There is a one-to-one mapping between the entities since the same column is the primary and foreign keys.

Why Use a Shared Primary Key?

- It guarantees that for every principal entity, there is exactly one dependent entity and vice versa.
- You only need one column in the dependent table to serve as the primary and foreign keys.
- This can simplify queries and reduce redundancy, as only one key column is used to reference the relationship.

How to Implement a Shared Primary Key in EF Core:

Let's assume we have two entities, User (principal entity) and Passport (dependent entity), and we want to create a One-to-One relationship where the primary key of the Passport entity is also its foreign key to the User entity. I mean, we are going to work with the same example.

With Data Annotation:

Let us first see how to implement it with Data Annotation, and then we will see how to implement it using Fluent API Configuration. With Data Annotation, modify the User and Passport entities as follows:

User Entity:

Please modify the User entity as follows. The User entity has a UserId primary key.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class User
    {
        public int UserId { get; set; } // Primary Key
        public string Username { get; set; }
        public Passport Passport { get; set; } // Navigation Property to
Passport
    }
}
```

Passport Entity:

Please modify the Passport entity as follows. The Passport entity will use the same UserId as its primary and foreign keys.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    public class Passport
    {
        [Key] // Defines UserId as the Primary Key for Passport
        [ForeignKey("User")] // Defines UserId as the Foreign Key to the
User entity
        public int UserId { get; set; } // Primary Key & Foreign Key
        public string PassportNumber { get; set; }
        public User User { get; set; } // Navigation Property to User
    }
}
```

In this example:

- The UserId property in the Passport entity is marked with [Key] to indicate that it is the passport's primary key.
- The same UserId property is marked with [ForeignKey("User")] to indicate it as the foreign key to the User entity.
- Thus, the Passport entity's primary key (UserId) is the foreign key that links back to the User entity.

Modifying the Context Class:

Next, modify the EFCoreDbContext class as follows:

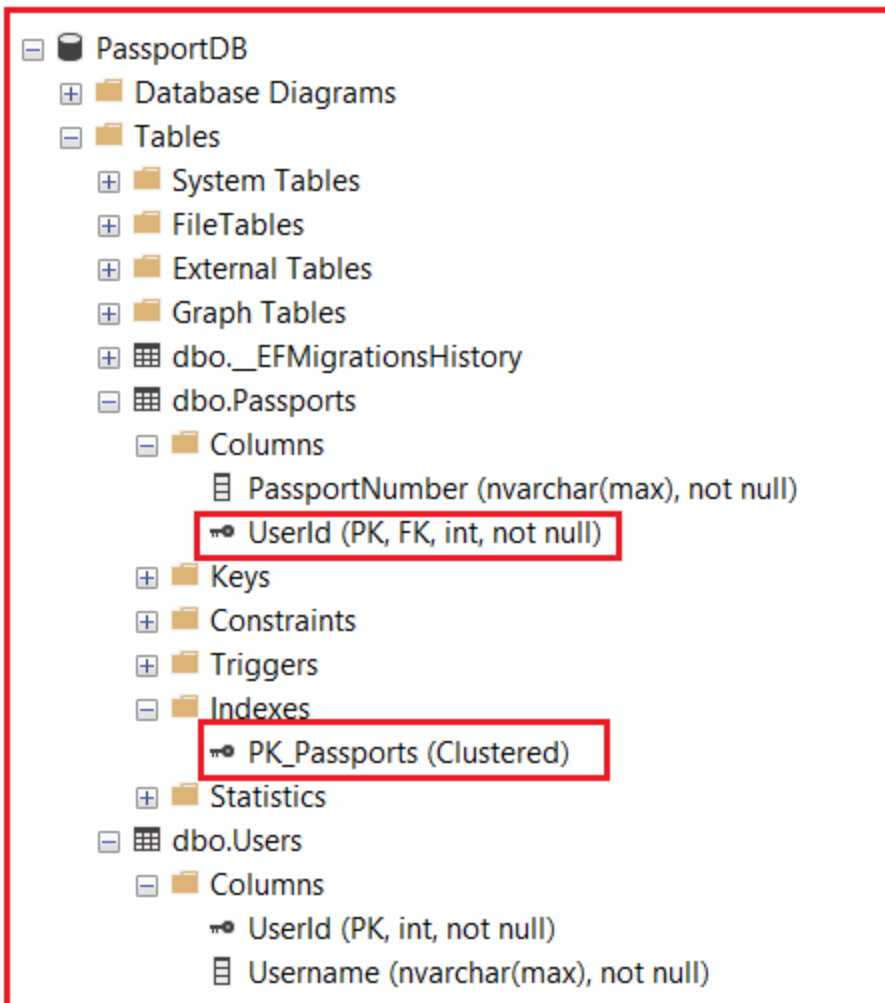
```

using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            //Configuring the Connection String
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=PassportDB;Trusted_Connection=True;TrustServerCertificate=True;");
        }
        public DbSet<User> Users { get; set; }
        public DbSet<Passport> Passports { get; set; }
    }
}

```

Generating Migration:

With the above changes, open the Package Manager Console and Execute the Add-Migration and Update-Database commands. Once the commands are executed, please verify the database. You should see the following:



As you can see, the UserId column in the Passports table has become the Primary Key, and the foreign key is pointing to the Users table. Also, notice that we don't have a Unique Non-Clustered Index now, which makes sense because we have a Clustered Index on the UserId column of the Passport table.

Shared Primary Key with Fluent API Configuration in EF Core

If you prefer to use the Fluent API to configure the shared primary key, you can do it using the `OnModelCreating` method of the `DbContext` class. So, for a better understanding, please modify the `EFCoreDbContext` class as follows:

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            //Configuring the Connection String
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=PassportDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            // Configure the User entity
            modelBuilder.Entity<User>()
                .HasOne(u => u.Passport) // User has one Passport
                .WithOne(p => p.User) // Passport has one related User
                .HasForeignKey<Passport>(p => p.UserId); // The UserId is the foreign key to the User entity
            // Configure the Passport entity
            modelBuilder.Entity<Passport>()
                .HasKey(p => p.UserId); // Set UserId as the Primary Key for the Passport table
        }
        public DbSet<User> Users { get; set; }
        public DbSet<Passport> Passports { get; set; }
    }
}
```

Explanation:

- `HasOne(u => u.Passport)` and `WithOne(p => p.User)` establish the One-to-One relationship between User and Passport.
- `HasForeignKey<Passport>(p => p.UserId)` specifies that `UserId` in the Passport entity is the foreign key to the User entity.
- `HasKey(p => p.UserId)` defines the same `UserId` as the primary key of the Passport entity.

With the above changes, you can remove the Data annotation attributes from the Entities, generate the Migration, and Update the database. You should see the expected One-to-One Relationship between the User and the Passport with a shared Primary key.

When to Use One-to-One Relationships in Entity Framework Core

One-to-one relationships are suitable when:

- Each entity must be linked to exactly one other entity. For example, a user has only one passport and vice versa.

- Entities need to be split into different tables for better organization. Sometimes, separating related data into different tables can help organize the database schema more efficiently, especially if one table contains sensitive information (e.g., User vs. Passport).
- When you need a unique foreign key relationship (i.e., a foreign key must also be a primary key).

In the next article, I will discuss [how to Configure One-to-Many Relationships in Entity Framework Core](#) with Examples. In this article, I explain **how to Configure One-to-One Relationships Between Entities in Entity Framework Core** with Examples. I hope you enjoyed this article on Configuring One-to-One Relationships using EF Core.